SOCKET PROGRAMMING

Absolute Basics

Prepared by -
Niloy Deb Roy
Contractual Lecturer
BRAC University

What is Socket Programming?

- The term *socket programming* refers to writing programs that execute across multiple computers in which the devices are all connected to each other using a network.
- Socket programming can be connection-oriented or connection-less.
- Python uses it's socket library to implement socket programming.
- Transmission Control Protocol (TCP) is used as a default protocol.

Client-side Socket Programming

- We use the socket object to make connection to the server, send data to and receive data from the server.
- Steps involved in a typical communication with the server –
  1. First, socket library is imported and *socket()* object is created.
  2. The client initiates connection to a server specified by hostname/IP address and port number using *connect((hostname/ip_address, port_number))* method.
  3. Send data to the server using *send(message)* method.
  4. Read data from the server using *recv(buffer_size)* method.
  5. Close the connection using *close()* method.

Client-side Socket Programming

- Method Description:
  - **socket.socket(socket.AF_INET, socket.SOCK_STREAM):**

    Here, AF_INET refers to the address-family ipv4.

    The SOCK_STREAM means connection-oriented TCP protocol.
  - **connect((Hostname/Ip Address, Port Number)):**

    Here, the tuple takes hostname or ip address of the server and the port number.
  - **send(message):**

    Here, the given message needs to be encoded using an encoding format. E.g., `message.encode('utf-8')`.
  - **recv(buffer_size):**

    Here, the return value is a bytes object representing the data received. It needs to be decoded using appropriate format. E.g., `recv(2048).decode('utf-8')`.

Server-side Socket Programming

- We use the socket object to make connection with the client, send data to and receive data from the client.
- Steps involved in a typical communication with the client–
  1. First, socket library is imported and *socket()* object is created.
  2. Bind the server socket with hostname/IP address and port number using *bind((hostname/ip_address, port_number))* method.
  3. Listen for a connection from the client using *listen()* method and accept it using *accept()* method. This results in a client socket being created for the connection.
  4. Use the client socket to send data to the client and receive data from the client.
  5. Close the connection using *close()* method.

Server-side Socket Programming

- Method Description:

  - listen(backlog):

  Enable a server to accept connections. The backlog specifies the number of unaccepted connections that the system will allow before refusing new connections. If not specified, a default reasonable value is chosen.

  - accept():

  Accept a connection. The return value is a pair *(conn, address)* where conn is a new socket object usable to send and receive data on the connection, and address is the address bound to the socket on the other end of the connection.

Socket Programming

- Other Methods:
  - **socket.gethostname():**

    This returns a string containing the *hostname* of the machine where the Python interpreter is currently executing.
  - **socket.gethostbyname(host_name):**

    Translate a host name to IPv4 address format. The IPv4 address is returned as a string, such as '100.50.200.5'. If the host name is an IPv4 address itself it is returned unchanged.

How it works

localhost:8085

Client

localhost:6666

Server

```
host= socket.gethostbyname(socket.gethostname())
port = 6666
addr = (host, port)
```

```
host= socket.gethostbyname(socket.gethostname())
port = 6666
addr = (host, port)
```

How it works

localhost:8085

Client

```
client = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
```

localhost:6666

Server

```
server = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)

server.bind(addr)
```

How it works

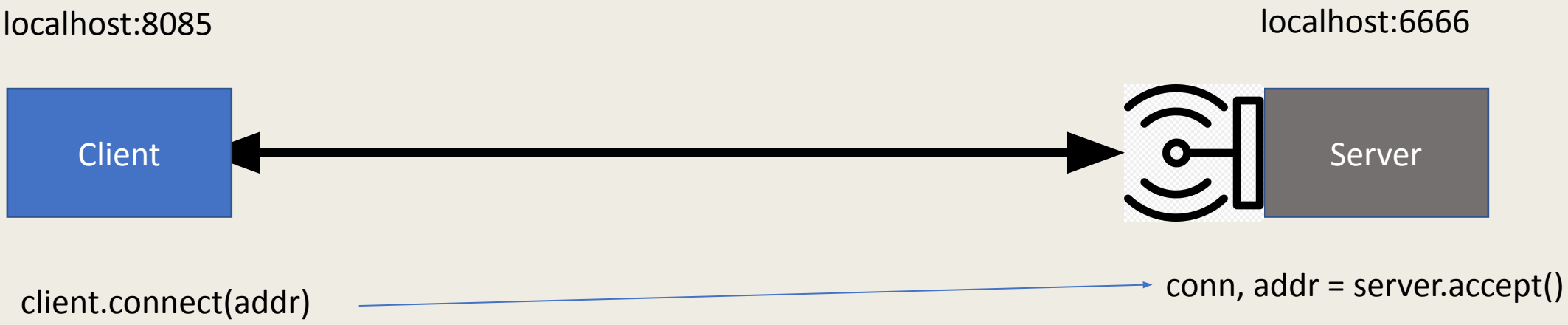
localhost:8085

Client

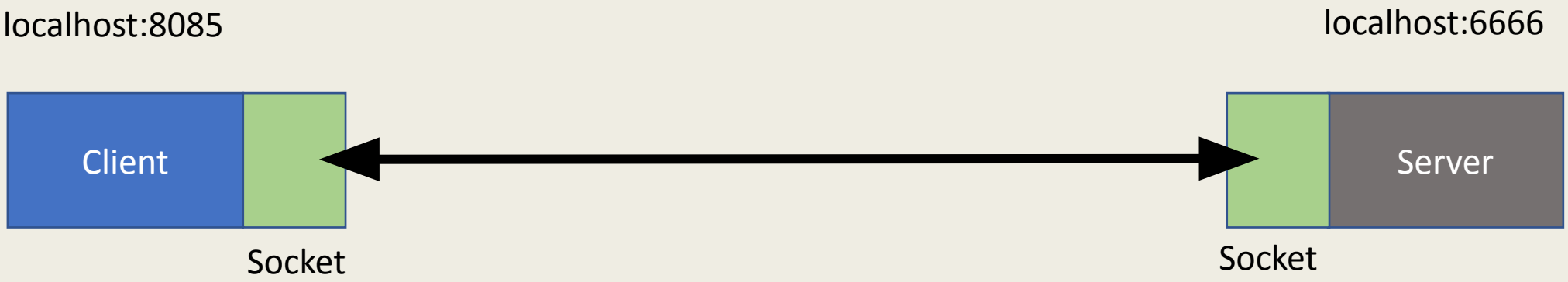
localhost:6666

server.listen()

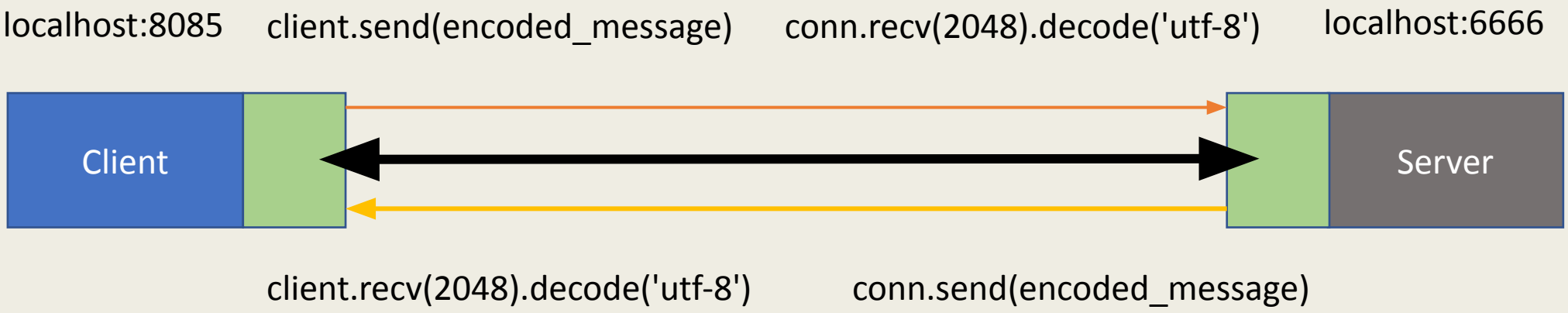
How it works


localhost:8085
localhost:6666
Client
Server
client.connect(addr)
conn, addr = server.accept()

How it works


localhost:8085
localhost:6666
Client
Server
Socket
Socket

How it works


localhost:8085
client.send(encoded_message)
conn.recv(2048).decode('utf-8')
localhost:6666
Client
Server
client.recv(2048).decode('utf-8')
conn.send(encoded_message)

LET’S START CODING